L02: Backpropagation

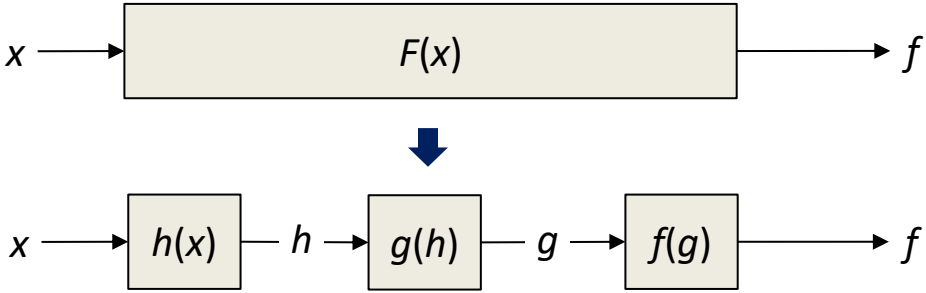
FICT, UTAR

Table of Contents

Chain Rule

Composite Function

- The forward propagation implements a composite function. A composite function is a function that is made up of a chain of smaller functions.



$$F(x) = (f \circ g \circ h)\,(x) = f[g[h(x)]]$$

where $(f \circ g \circ h)$ is executed in the order $h(x) \rightarrow g(h) \rightarrow f(g)$.

- Different order leads to a different composite function.

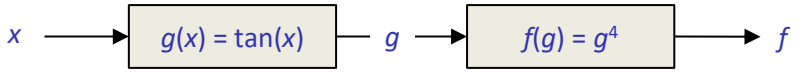
$$(f \circ g \circ h)\,(x) \neq (h \circ g \circ f)\,(x)$$

Example

Convert the following functions into their composite forms.

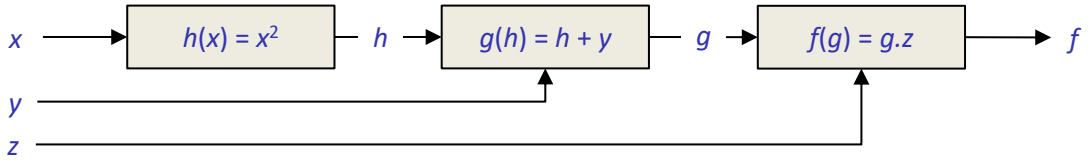
- $F(x) = \tan^4(x)$

$$F(x, y, z) = (f \circ g)(x)$$



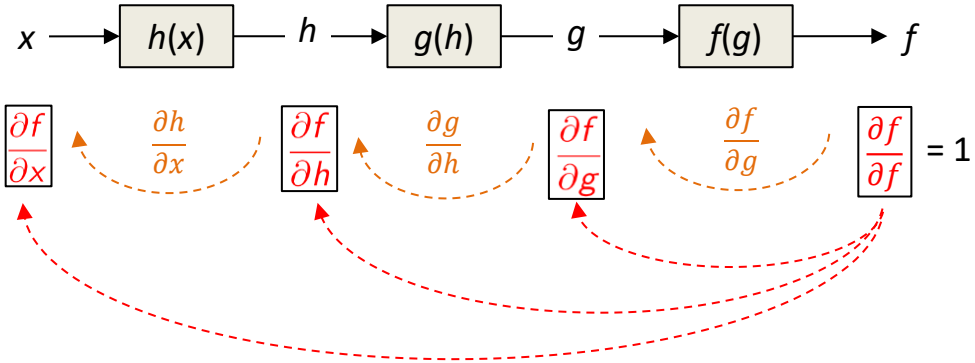
- $F(x) = (x^2 + y)z$

$$F(x, y, z) = (f \circ g \circ h)(x, y, z)$$



Local Gradient and Global Gradient

- The **local gradient** of a composite function is the gradient of a local function's output w.r.t. its input.
- The **global gradient** of a composite function is the gradient of the final output w.r.t. the signal in the function.

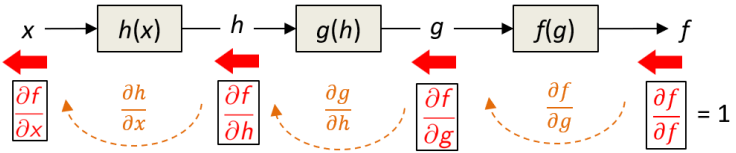


Notes:

- $\partial f/\partial g$ is read as the "*gradient of f w.r.t. x*" or the "gradient at x". Since there are multiple variables in a composite function, we are doing partial differentiation.
- Formally, gradient is for vectors. For scalar variable, we use the term derivative.

The Chain Rule

1. Given a composite function $F(x) = (f \circ g \circ h)(x)$, chain rule computes a global gradient by chaining the local gradients.



$$\frac{\partial f}{\partial g} = \frac{\partial f}{\partial g} = \boxed{\frac{\partial f}{\partial f} \cdot \frac{\partial f}{\partial g}}$$

$$\frac{\partial f}{\partial h} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial h} = \boxed{\frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial h}}$$

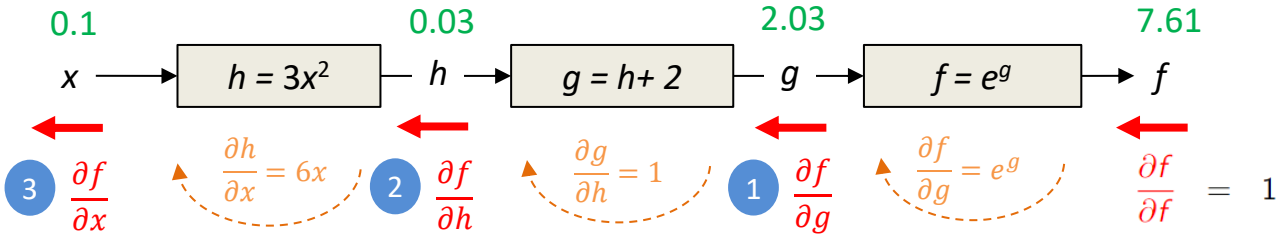
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial h} \cdot \frac{\partial h}{\partial x} = \boxed{\frac{\partial f}{\partial h} \cdot \frac{\partial h}{\partial x}}$$

2. The chain rule can be generalized to the following rule:

Global gradient of signal	=	Upstream global gradient	×	Local gradient of signal

Example

Given $F(x) = e^{3x^2+2}$, compute $F'(x = 0.1)$ using the chain rule.



1 $\frac{\partial f}{\partial g} = \frac{\partial f}{\partial f} \cdot \frac{\partial f}{\partial g} = \frac{\partial f}{\partial f} \times e^g = 1 \times 7.61 = 7.61$

2 $\frac{\partial f}{\partial h} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial h} = \frac{\partial f}{\partial g} \times 1 = 7.61 \times 1 = 7.61$

3 $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial h} \cdot \frac{\partial h}{\partial x} = \frac{\partial f}{\partial h} \times 6x = 7.61 \times 0.6 = 4.57$

Therefore, $F'(x = 0.1) = 4.57$

The Backpropagation Algorithm

The Backpropagation Algorithm

- Computes the gradient of the cost w.r.t. all parameters in the network.
- Commences in two steps:

Forward Propagation

- Computes the activation values layer by layer in the forward direction:

  $$input \rightarrow layer\ 1 \rightarrow \ldots \rightarrow output\ layer \rightarrow cost$$

- PyTorch builds the computational graph in the process.

Backpropagation

- Computes the gradient values layer by layer in the backward direction:

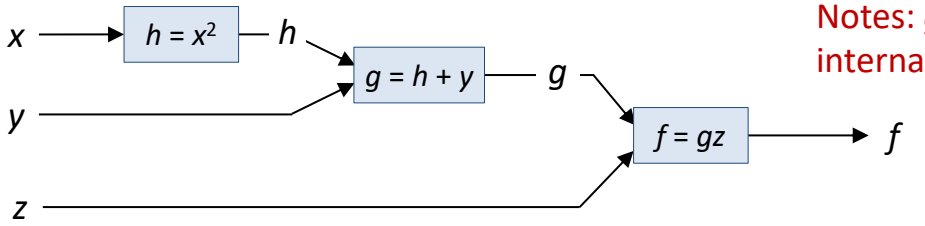
  $$Cost \rightarrow output\ layer \rightarrow \ldots \rightarrow layer\ 1 \rightarrow input$$

- PyTorch destroys the computational graph in the process.

Computational Graph

- Forward propagation implements the composite function by constructing a computational graph:

$$F(x) = (x^2 + y)z$$



Notes: *g* and *h* are internal signals

- Backpropagation computes the global gradients of the output *f* w.r.t. *all* signals (*x*, *y*, *z*, *p*, *q*) as it backpropagates through the graph.

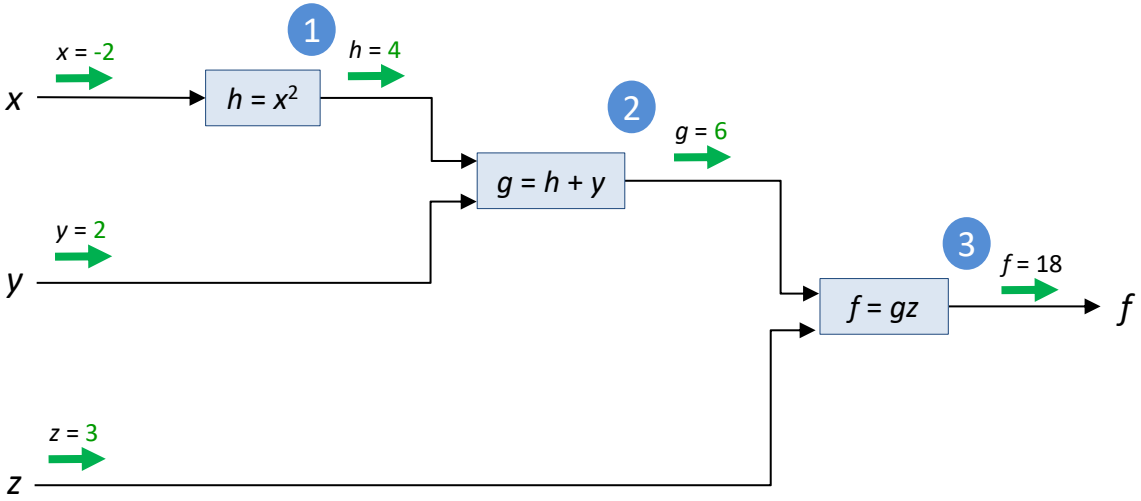
$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z} \quad \frac{\partial f}{\partial g} \quad \frac{\partial f}{\partial h}$$

Forward Propagation (Example)

- Example: $F(x) = (x^2 + y)z$

Given x = -2, y = 2, z = 3

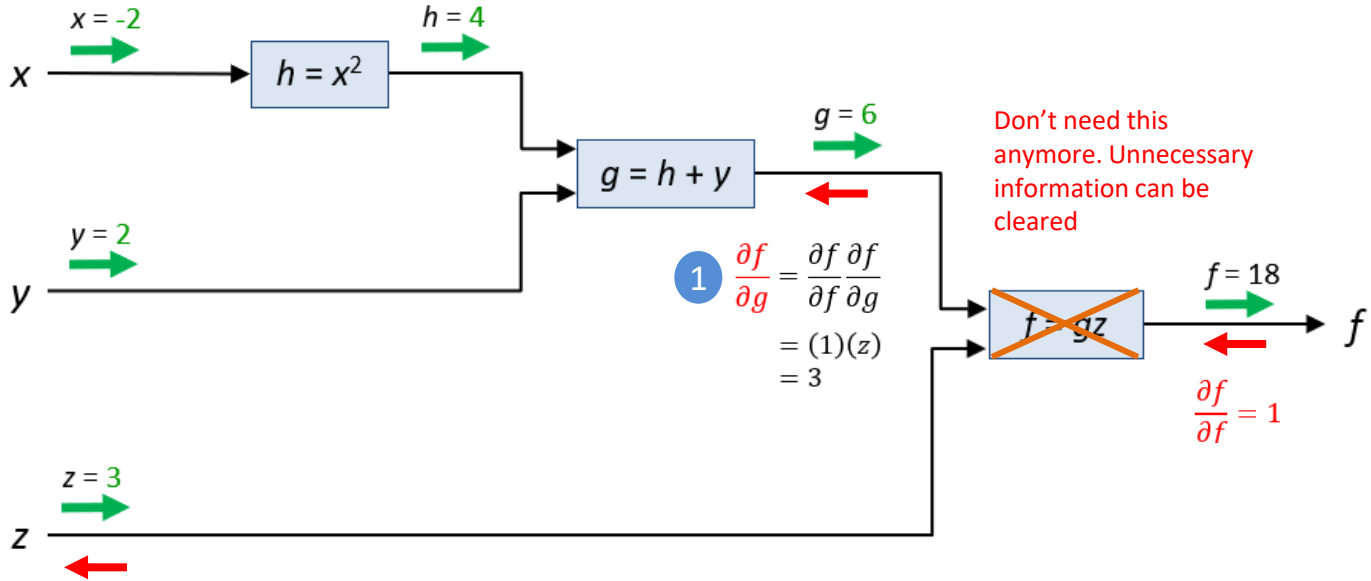
Forward propagation builds the computational graph dynamically in PyTorch.



Backpropagation (Example)

Global gradient = Upstream global gradient × Local gradient

- Given x = -2, y = 2, z = 3

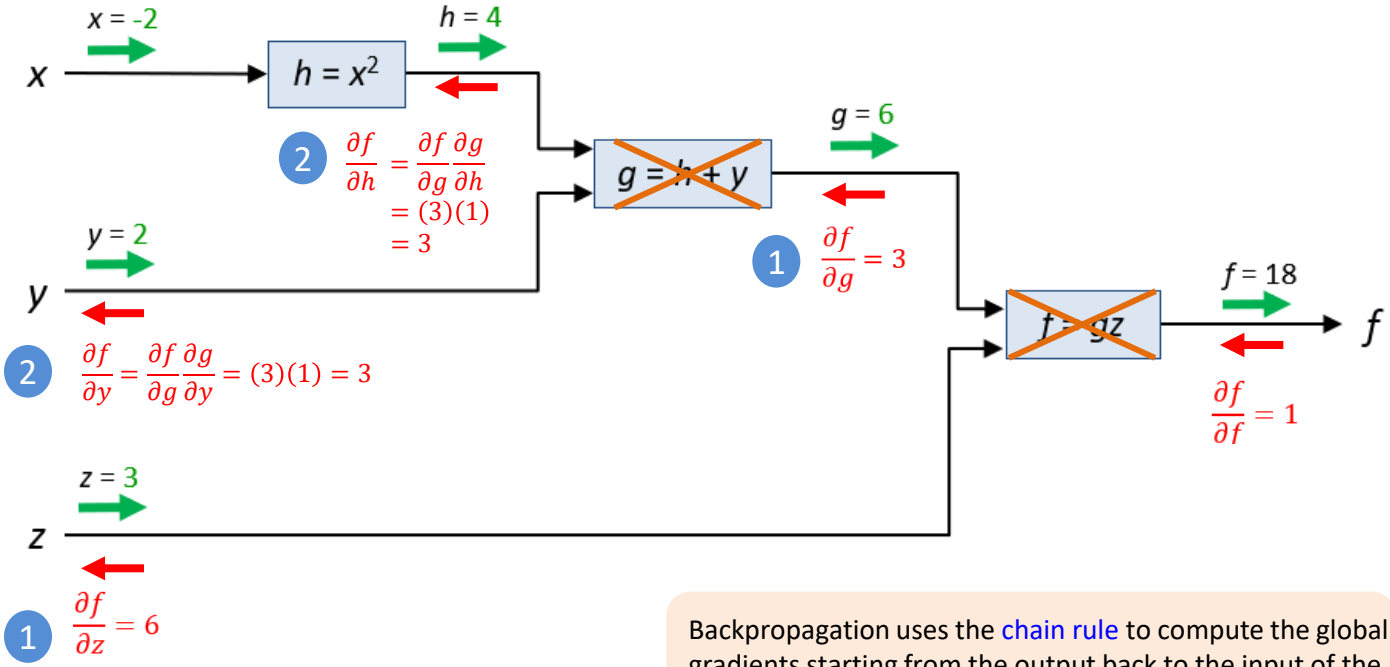


Backpropagation uses the chain rule to compute the global gradients starting from the output back to the input of the computational graph (backward direction).

Backpropagation (Example)

Global gradient = Upstream global gradient × Local gradient

- Given x = -2, y = 2, z = 3



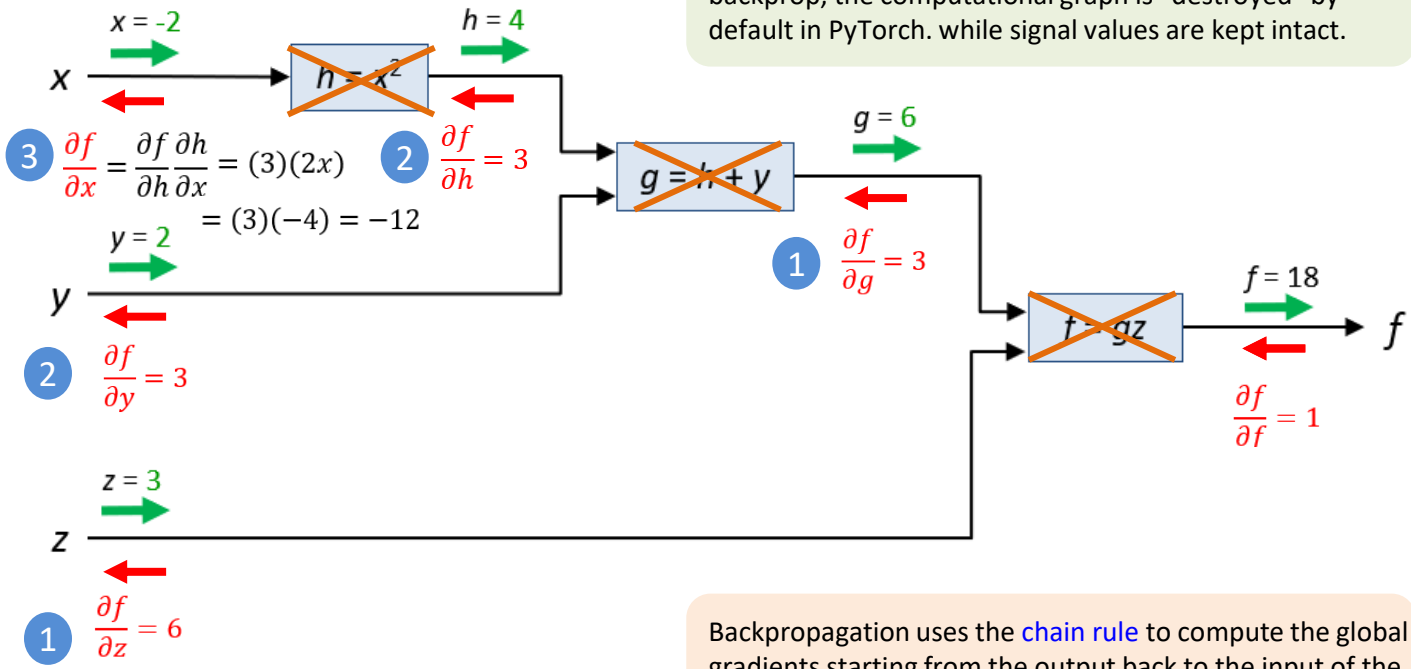
Backpropagation uses the chain rule to compute the global gradients starting from the output back to the input of the computational graph (backward direction).

Backpropagation (Example)

- Given x = -2, y = 2, z = 3

Global gradient = Upstream global gradient × Local gradient

Backpropagation computes all global gradients. After backprop, the computational graph is "destroyed" by default in PyTorch. while signal values are kept intact.



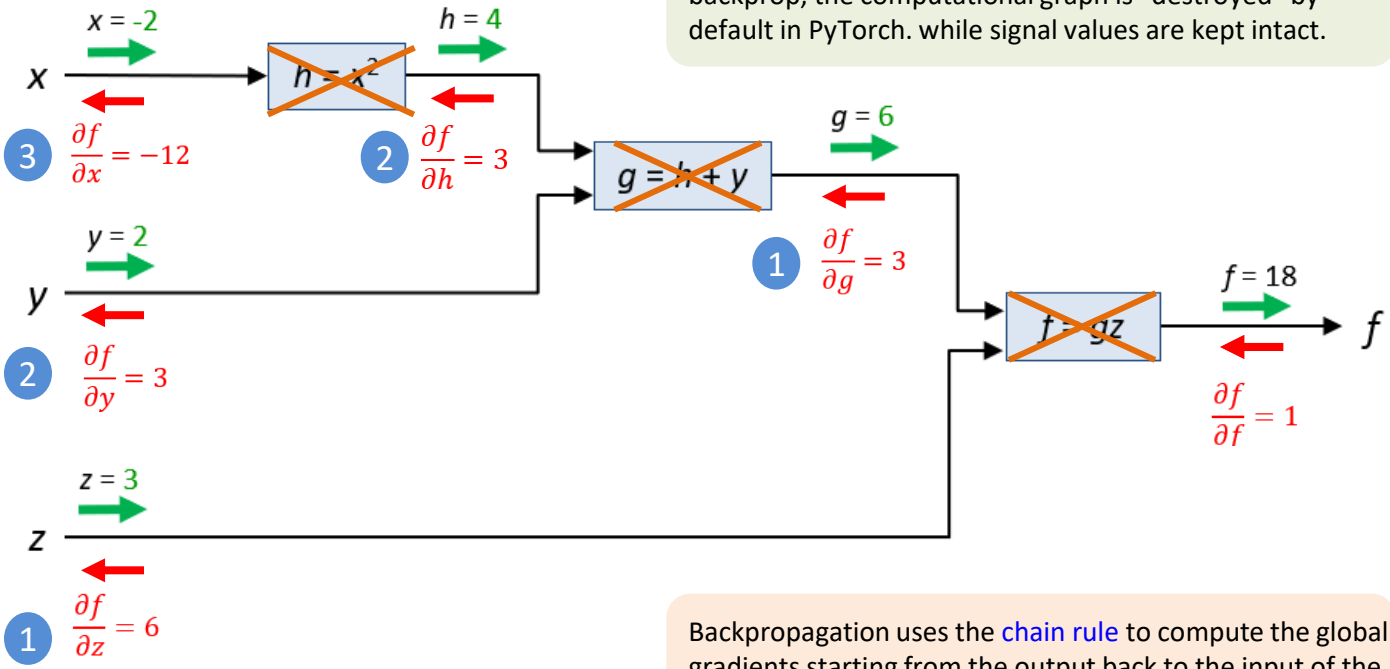
Backpropagation uses the chain rule to compute the global gradients starting from the output back to the input of the computational graph (backward direction).

Backpropagation example

- Given x = -2, y = 2, z = 3

Global gradient = Upstream global gradient × Local gradient

Backpropagation computes all global gradients. After backprop, the computational graph is "destroyed" by default in PyTorch. while signal values are kept intact.

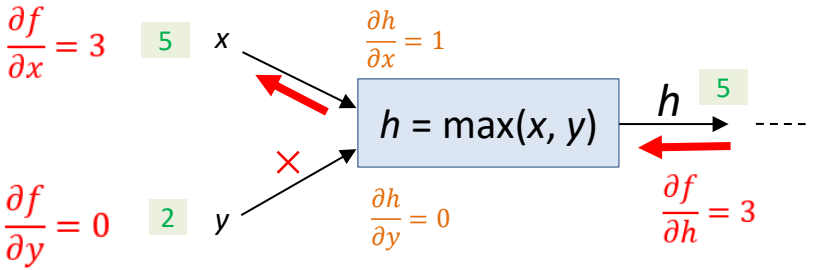


Backpropagation uses the chain rule to compute the global gradients starting from the output back to the input of the computational graph (backward direction).

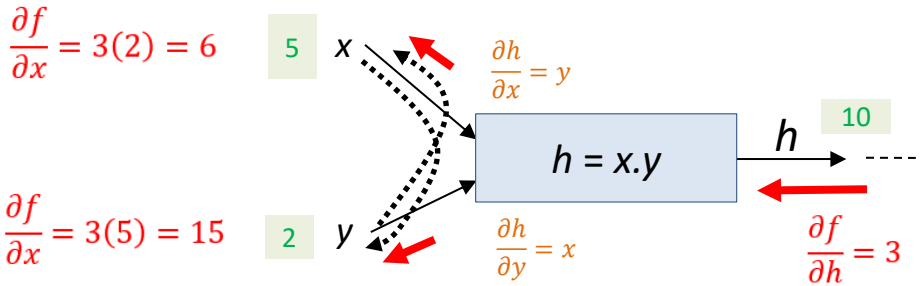
Backpropagating through Common Structures

Backpropagation through common structures

- max gate is a gradient router

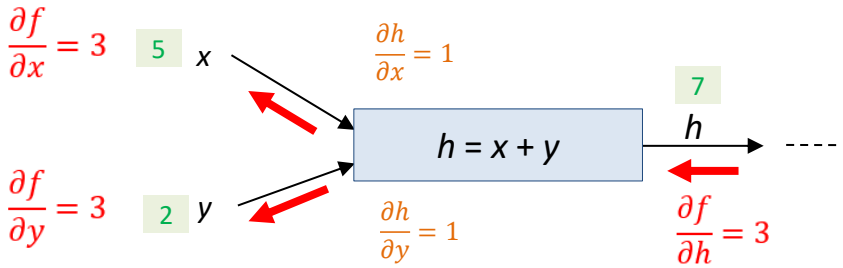


- mul gate is gradient switcher

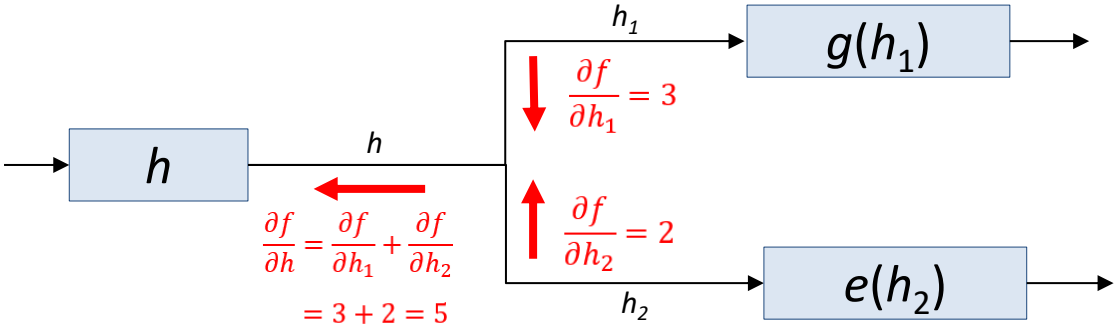


Backpropagation through common structures

- add gate: gradient distributor.



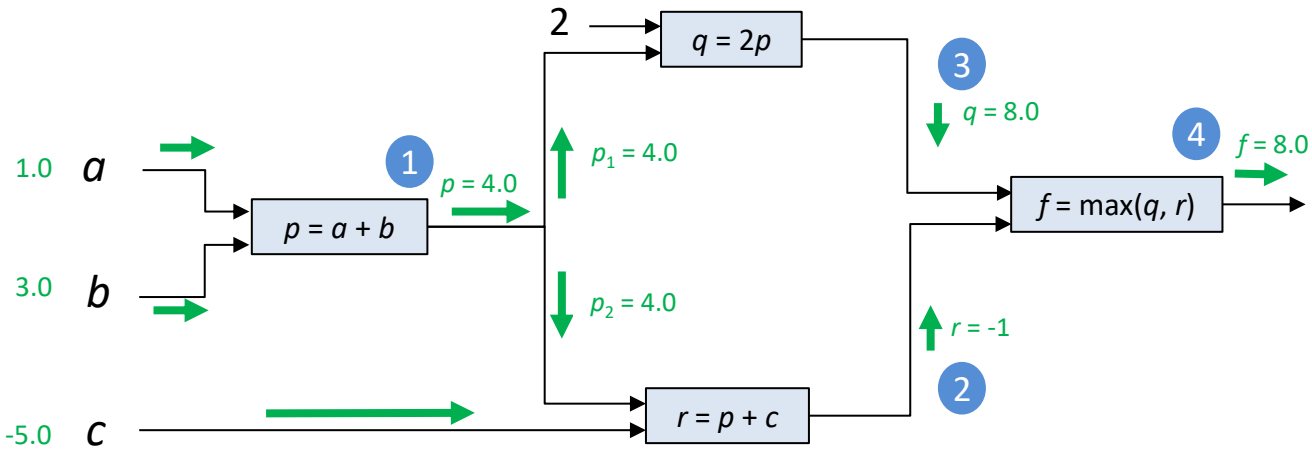
- For branches, simply add the upstream gradients from all branches.



Example

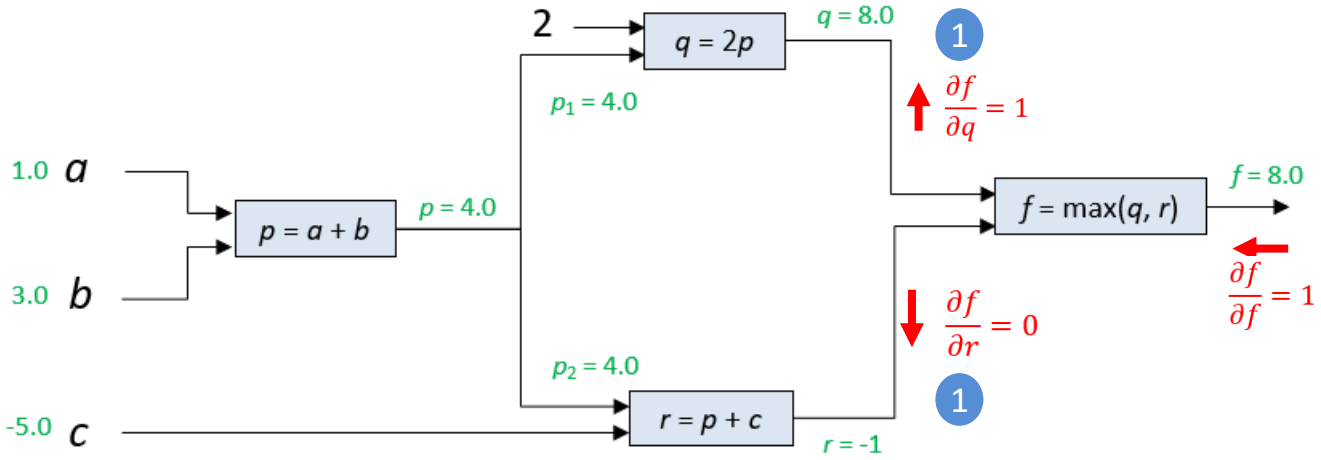
- Example:

$$F(a, b, c) = \max(2(a + b), a + b + c)$$



Backpropagation example (1 of 5)

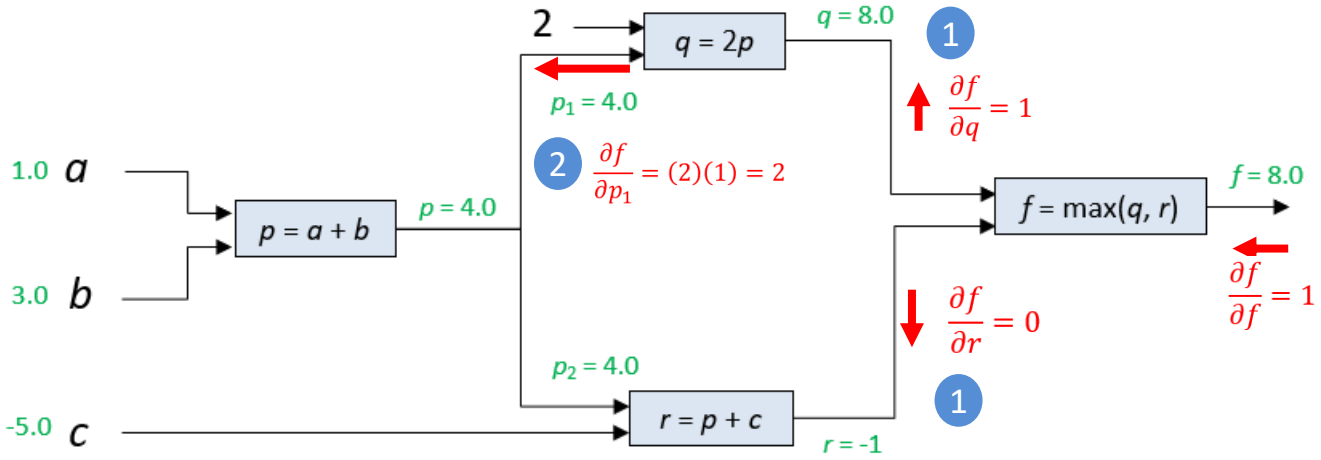
- Example:



Max gate is a gradient router.

Backpropagation example (2 of 5)

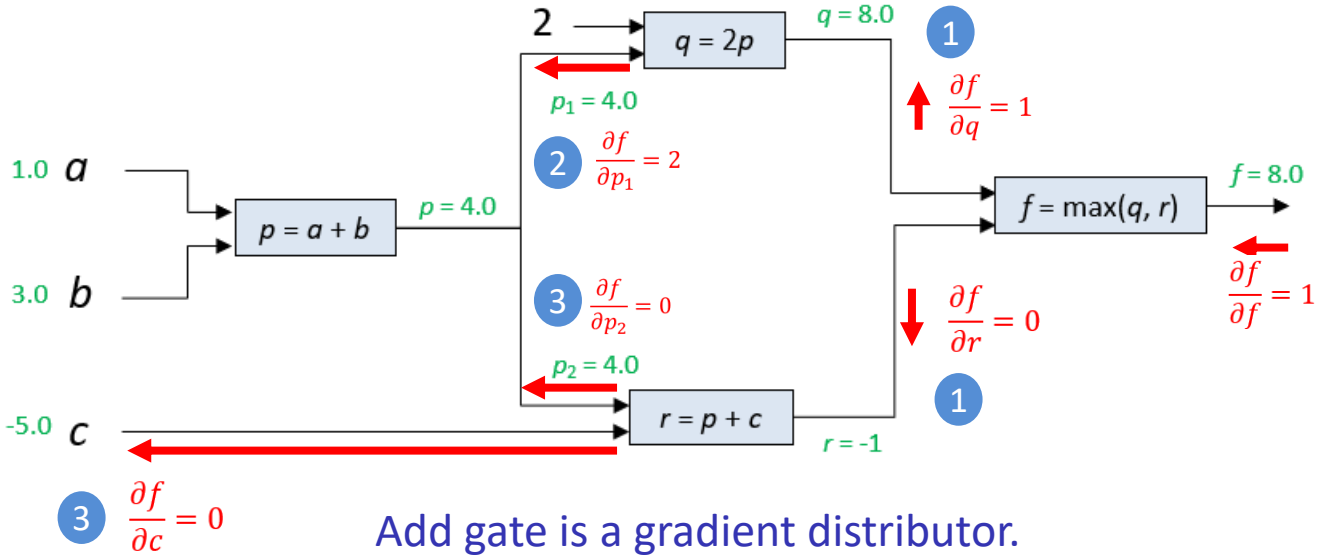
- Example:



Multiply gate is a gradient switcher.

Backpropagation example (3 of 5)

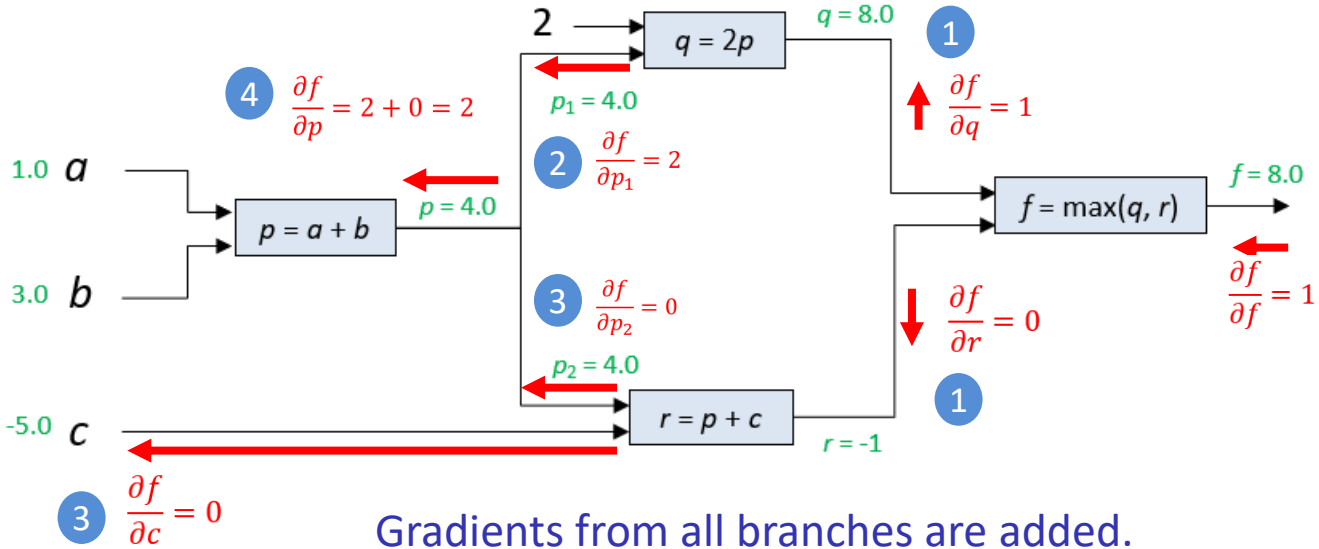
- Example:



Add gate is a gradient distributor.

Backpropagation example (4 of 5)

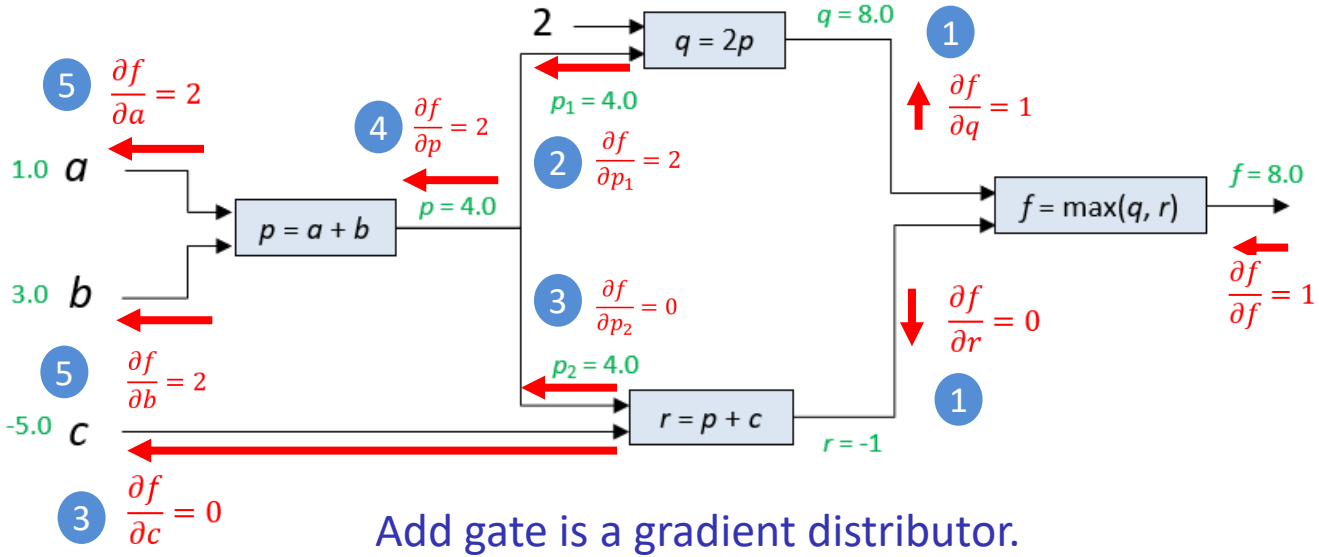
- Example:



Gradients from all branches are added.

Backpropagation example (5 of 5)

- Example:



Add gate is a gradient distributor.

Backpropagating in Logistic Regression

Backpropagation in Gradient Descent

- Given an arbitrary function, we can represent it as a computational graph

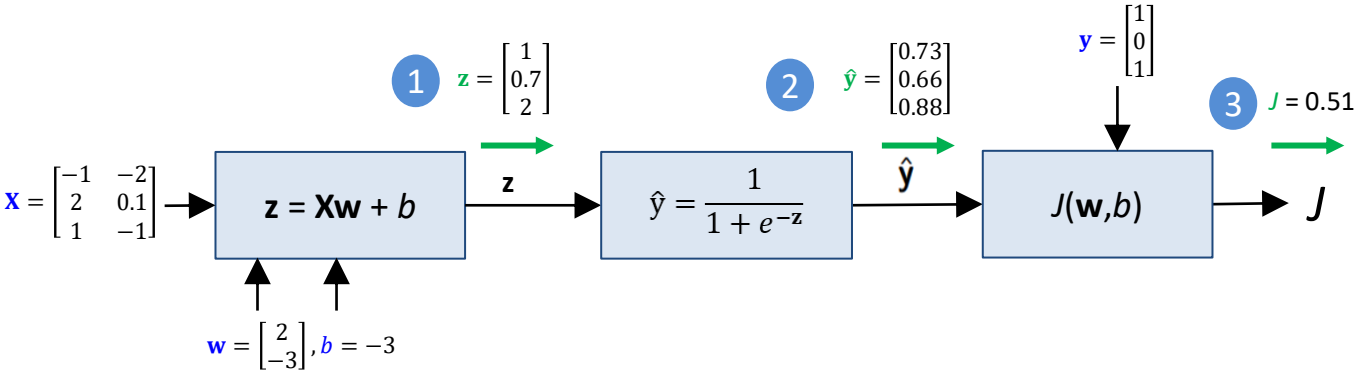
Algorithm 2 Batch Gradient Descent (BGD) for training LogReg

```
1: function BatchGradientDescent(X, y, α, iterations)
2:     Initialize w, b
3:     for t = 1 to num_iterations do
4:         ŷ ← forward_prop(X)
5:         J(w, b) ← cost_function(ŷ, y)
6:
7:         ∂J/∂w, ∂J/∂b ← backprop(J)
8:
9:         w ← w − α ∂J/∂w
10:        b ← b − α ∂J/∂b
11:    end for
12:    return x
13: end function
```

How to compute the gradient of network parameters? The Backpropagation Algorithm

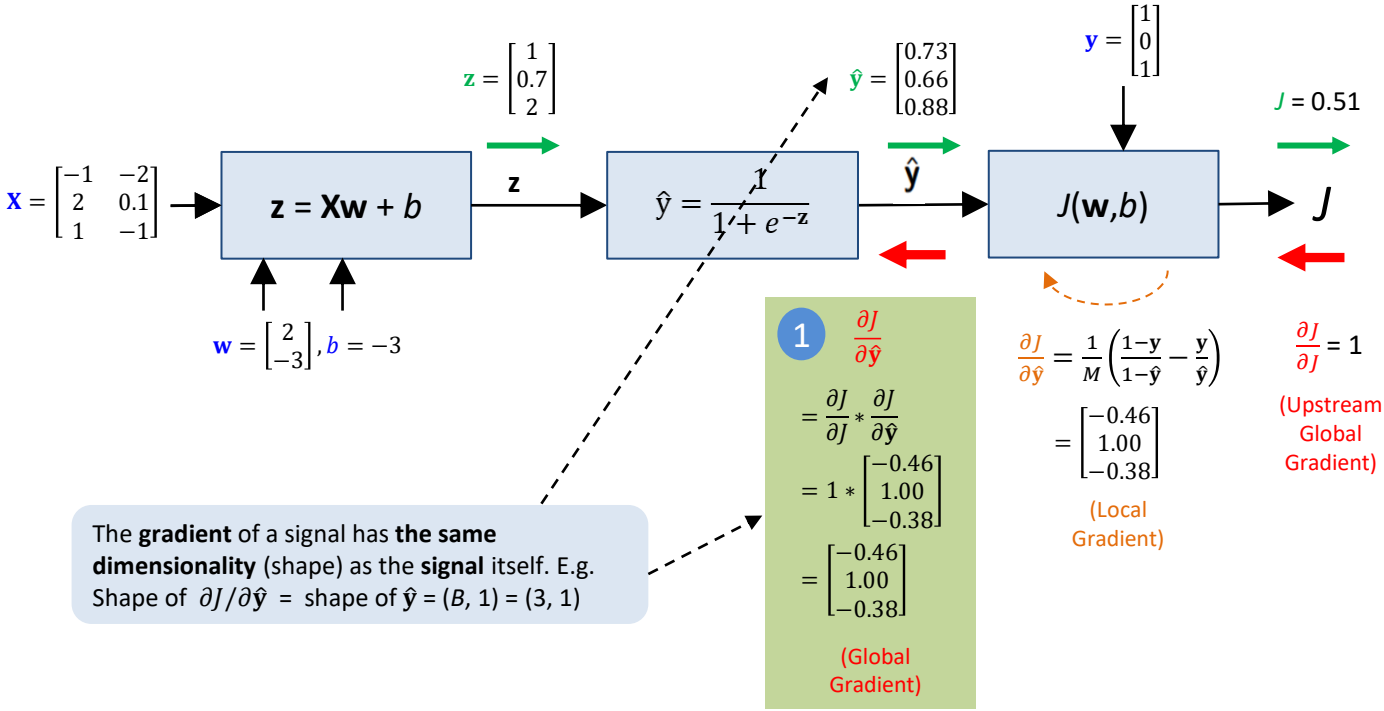
Example: Forward propagation

Given $\mathbf{w} = \begin{bmatrix} 2 \\ -3 \end{bmatrix}$, $b = -3$ and $\mathbf{X} = \begin{bmatrix} -1 & -2 \\ 2 & 0.1 \\ 1 & -1 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$



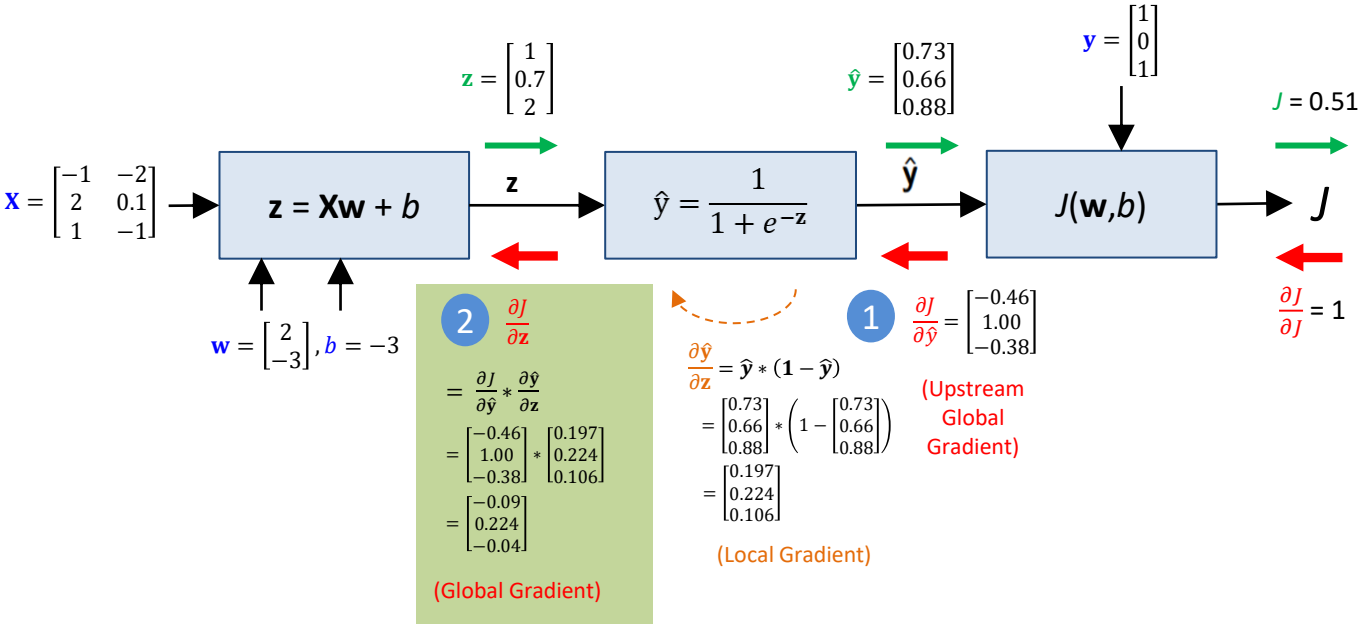
Example: Backpropagation (cost layer)

Given $\mathbf{w} = \begin{bmatrix} 2 \\ -3 \end{bmatrix}$, $b = -3$ and $\mathbf{X} = \begin{bmatrix} -1 & -2 \\ 2 & 0.1 \\ 1 & -1 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$



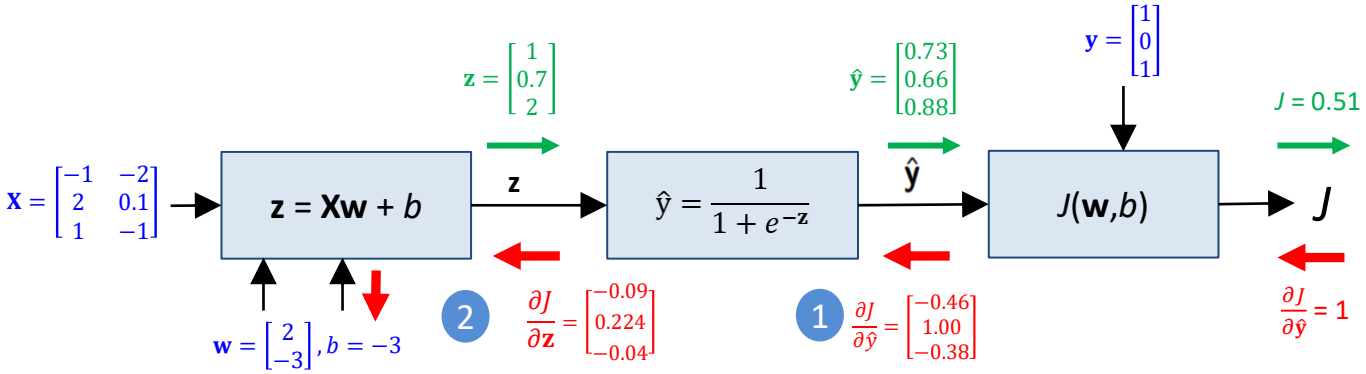
Example: Backpropagation (Activation layer)

Given $\mathbf{w} = \begin{bmatrix} 2 \\ -3 \end{bmatrix}$, $b = -3$ and $\mathbf{X} = \begin{bmatrix} -1 & -2 \\ 2 & 0.1 \\ 1 & -1 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$



Example: Backpropagation (Summation layer)

Given $\mathbf{w} = \begin{bmatrix} 2 \\ -3 \end{bmatrix}$, $b = -3$ and $\mathbf{X} = \begin{bmatrix} -1 & -2 \\ 2 & 0.1 \\ 1 & -1 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$



(upstream global gradient)

3 $\frac{\partial J}{\partial b}$

$= \left(\frac{\partial \mathbf{z}}{\partial b}\right)^T \frac{\partial J}{\partial \mathbf{z}}$

$= \mathbf{1}^T \frac{\partial J}{\partial \mathbf{z}}$

$= \text{sum}\left(\frac{\partial J}{\partial \mathbf{z}}\right)$

$= 0.09$

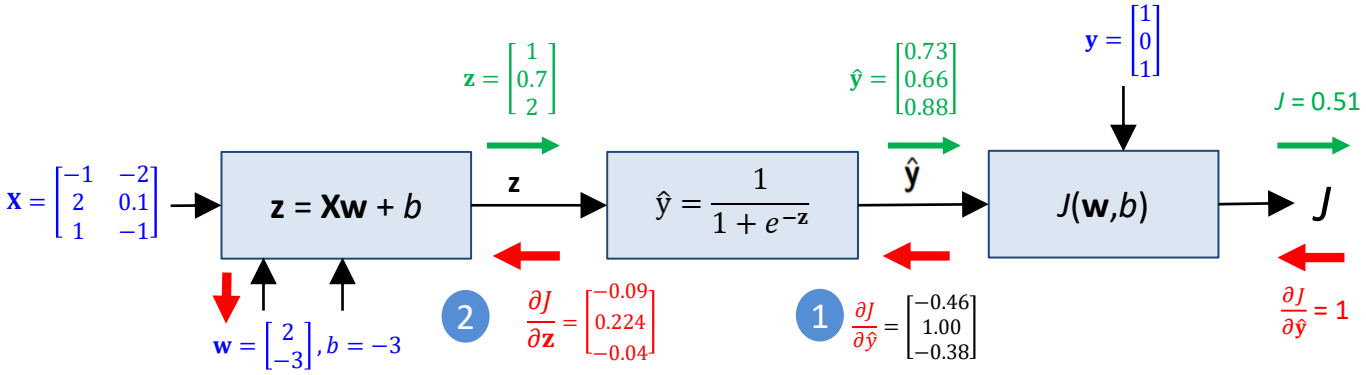
(Global Gradient)

$\frac{\partial \mathbf{z}}{\partial b} = \begin{bmatrix} \partial z^{(1)}/\partial b \\ \partial z^{(2)}/\partial b \\ \partial z^{(3)}/\partial b \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$

(local gradient)

Example: Backpropagation (Summation layer)

Given $\mathbf{w} = \begin{bmatrix} 2 \\ -3 \end{bmatrix}$, $b = -3$ and $\mathbf{X} = \begin{bmatrix} -1 & -2 \\ 2 & 0.1 \\ 1 & -1 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$



3 $\frac{\partial J}{\partial \mathbf{w}} = \left(\frac{\partial \mathbf{z}}{\partial \mathbf{w}}\right)^T \frac{\partial J}{\partial \mathbf{z}} = \mathbf{X}^T \frac{\partial J}{\partial \mathbf{z}} = \begin{bmatrix} 0.5 \\ 0.24 \end{bmatrix}$

(Global Gradient)

3 $\frac{\partial J}{\partial b} = 0.09$

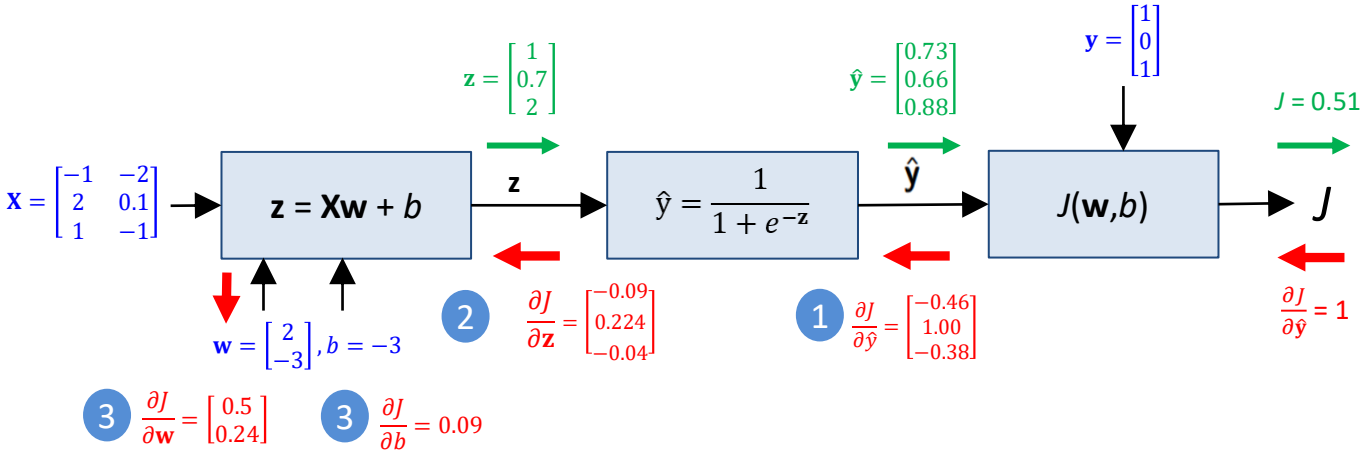
(upstream global gradient)

$\frac{\partial \mathbf{z}}{\partial \mathbf{w}} = \mathbf{X} = \begin{bmatrix} \partial z^{(1)}/\partial w_1 & \partial z^{(1)}/\partial w_2 \\ \partial z^{(2)}/\partial w_1 & \partial z^{(2)}/\partial w_2 \\ \partial z^{(3)}/\partial w_1 & \partial z^{(3)}/\partial w_3 \end{bmatrix}$

(Local Gradient)

Example: Backpropagation (Complete)

Given $\mathbf{w} = \begin{bmatrix} 2 \\ -3 \end{bmatrix}$, $b = -3$ and $\mathbf{X} = \begin{bmatrix} -1 & -2 \\ 2 & 0.1 \\ 1 & -1 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$



Gradient Descent in Logistic Regression

Backpropagation Algorithm in Logistic Regression

- Batch Gradient Descent (BGD, same as previous lecture):

Algorithm Batch Gradient Descent (BGD) for training LogReg

```
1:  function BatchGradientDescent(X, y, α, iterations)
2:      Initialize w, b                                   [1] Initialize
3:      for t = 1 to num_iterations do
4:          ŷ ← forward(X, w, b)                          [2] Forward prop to predict output and
5:          J(w, b) ← compute_cost(ŷ, y)                      compute cost
6:          ∂J/∂w, ∂J/∂b ← backward(X, y, ŷ)              [3] Backprop to get parameter gradients
7:          w ← w − α ∂J/∂w                               [4] Update network parameters
8:          b ← b − α ∂J/∂b
9:      end for
10:     return x
11: end function
```

Backpropagation Algorithm in Logistic Regression

- Forward propagation:

Algorithm Forward Propagation for Logistic Regression

1: **function** FORWARD($\mathbf{X}$, $\mathbf{w}$, b)
2: $\quad \mathbf{z} = \mathbf{X}\mathbf{w} + b$
3: $\quad \hat{\mathbf{y}} = \frac{1}{1+e^{-\mathbf{z}}}$
4: $\quad$ **return** $\hat{\mathbf{y}}$
5: **end function**

1. Summation phase (Linear)
2. Activation phase (sigmoid)

- Cost function:

Algorithm Binary Cost Entropy

1: **function** COMPUTE_COST($\hat{\mathbf{y}}$, $\mathbf{y}$)
2: $\quad J = \text{mean}(-\mathbf{y}\ln(\hat{\mathbf{y}}) - (1-\mathbf{y})\ln(1-\hat{\mathbf{y}}))$
3: $\quad$ **return** J
4: **end function**

Backpropagation Algorithm in Logistic Regression

- Backpropagation

Algorithm Backpropagation for Logistic Regression

1: **function** BACKWARD($\mathbf{X}$, $\mathbf{y}$, $\hat{\mathbf{y}}$)
2: $B, F \leftarrow$ shape of $\mathbf{X}$
3: $\frac{\partial J}{\partial \hat{\mathbf{y}}} = \frac{1}{M}\left(\frac{(1-\mathbf{y})}{(1-\hat{\mathbf{y}})} - \frac{\mathbf{y}}{\hat{\mathbf{y}}}\right)$ [1] Cost Function
4: $\frac{\partial J}{\partial \mathbf{z}} = \frac{\partial J}{\partial \hat{\mathbf{y}}} \cdot \hat{\mathbf{y}} \cdot (1 - \hat{\mathbf{y}})$ [2] Activation layer
5: $\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T \frac{\partial J}{\partial \mathbf{z}}$
6: $\frac{\partial J}{\partial b} = \mathbf{1}^T \frac{\partial J}{\partial \mathbf{z}}$ [3] Summation layer
7: **return** $\frac{\partial J}{\partial \mathbf{w}}, \frac{\partial J}{\partial b}$ Return the gradients
8: **end function**

Minibatch Gradient Descent (MGD)
and
Stochastic Gradient Descent (SGD)

Issues with BGD

- BGD needs to computes the activation and gradient of all samples for every single parameter update.
- When number of samples M is huge (say 1 million), can become memory and computationally intensive.

Algorithm Forward Propagation for Logistic Regression

function FORWARD_LOGREG($\mathbf{X}$, $\mathbf{w}$, b)
  $\mathbf{z} = \mathbf{X}\mathbf{w} + b$ ◄------- Memory and computation
  $\hat{\mathbf{y}} = \frac{1}{1+e^{-\mathbf{z}}}$ intensive when M is huge.
  return $\hat{\mathbf{y}}$
end function

Algorithm Backpropagation for Logistic Regression

function BACKWARD($\mathbf{X}$, $\mathbf{y}$, $\hat{\mathbf{y}}$)
  $B, F \leftarrow$ shape of $\mathbf{X}$
  $\frac{\partial J}{\partial \hat{\mathbf{y}}} = \frac{1}{M}\left(\frac{(1-\mathbf{y})}{(1-\hat{\mathbf{y}})} - \frac{\mathbf{y}}{\hat{\mathbf{y}}}\right)$
  $\frac{\partial J}{\partial \mathbf{z}} = \frac{\partial J}{\partial \hat{\mathbf{y}}} \cdot \hat{\mathbf{y}} \cdot (1 - \hat{\mathbf{y}})$
  $\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T \frac{\partial J}{\partial \mathbf{z}}$ ◄------- Memory and computation
  $\frac{\partial J}{b} = \mathbf{1}^T \frac{\partial J}{\partial \mathbf{z}}$ intensive when M is huge.
  return $\frac{\partial J}{\partial \mathbf{w}}, \frac{\partial J}{\partial b}$
end function

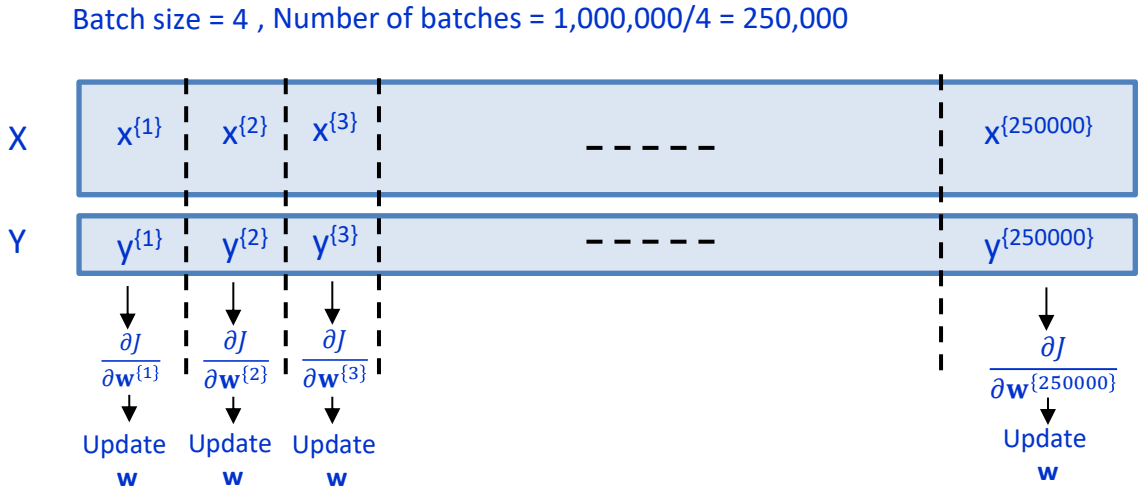
Mini-batch Gradient Descent (MGD)

- Strategy:
  - Split the training into mini-batches
  - Compute the gradient using batch samples
- Consider a dataset of 1 million samples:

Notation

$x^{(i)}$: sample i

$x^{\{t\}}$: batch t



- MGD updates w and b 250,000 times in one epoch whereas BGD updates only once.

Mini-batch Gradient Descent (MGD) and Stochastic Gradient Descent (SGD)

- The mini-batch gradient descent (MGD):

Algorithm MiniBatch Gradient Descent (MGD)

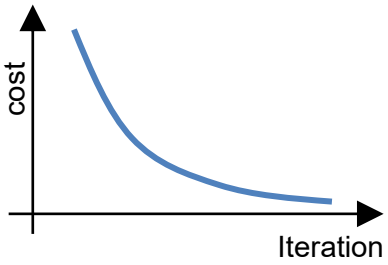
```
function MiniGradientDescent(X, y, batch_size, α, iterations)
    Initialize w, b
    for e = 1 to num_epochs do                                   (epoch)
        Shuffle X and y                                          (Shuffle before starting the epoch to get better performance)
        for t = 1 to num_batches do                              (batch iteration)
            X{t}, y{t} ← Get a batch of batch_size samples       (Get the batch data)
            ŷ{t} ← forward(X{t}, w, b)
            J{t} ← compute_cost(ŷ{t}, y{t})
            ∂J{t}/∂w, ∂J{t}/∂b ← backward(X{t}, y{t}, ŷ{t})     (Estimate the gradient based on batch data only)
            w ← w − α ∂J{t}/∂w
            b ← b − α ∂J{t}/∂b
        end for
    end for                                                      (1 epoch is when all training data is used once)
    return w, b
end function
```

- When the batch size B = 1, the algorithm is called Stochastic Gradient Descent (SGD).

Notes: Most online tutorials used SGD and MGD interchangeably.

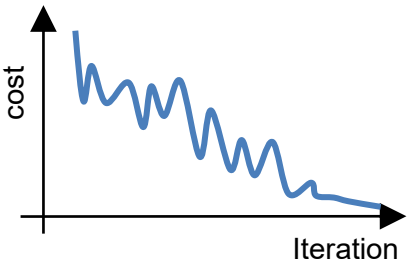
Training with mini batch gradient descent

Batch gradient descent



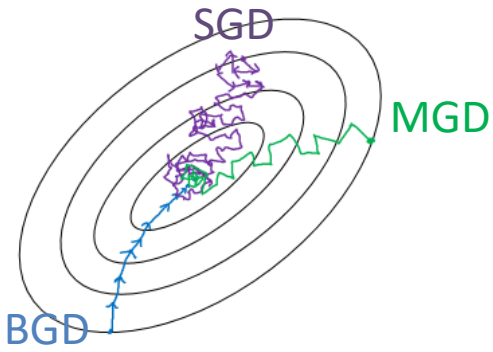
- Cost decreases smoothly and steadily
- Uses the full dataset
- Gradient has low variance (precise)

Mini-batch gradient descent



- Cost is noisy but trends downward
- Gradient is an estimate (higher variance)
- Smaller batches give implicit regularization

BGD vs MGD vs SGD



- BGD: smooth updates
- SGD/Mini-batch: noisy updates
- Noise helps find flatter minima and generalize better

Batch Gradient Descent	Mini-batch Gradient Descent	Stochastic Gradient Descent
• $B = M$ • Full vectorization • 1 update per epoch • High computation per epoch	• $1 < B < M$ • Partial vectorization • Multiple updates per epoch • Best practical choice	• $B = 1$ • No vectorization • Most updates per epoch • Noisy but fast learning

Guideline to setting the batch size

- Traditionally use powers of two (e.g., 4, 8, 16, 32) but has little impact on model performance.
- **Smaller batch**: noisier gradients, acts as regularization, better generalization
- **Larger batch**: better parallelization, faster per epoch but may need more epochs to converge.
- Practical tips:
  - **Small dataset** $\rightarrow$ smaller batch (reducing overfitting)
  - **Large models / large dataset** $\rightarrow$ large batch for pretraining but small batch for finetuning.
    E.g., RoBERTa: batch size = 8K for pre-training and 16/32/48 for fine-tuning
  - **Contrastive models** $\rightarrow$ large batch for better negative sampling
    E.g., Clip: batch size = 1K or 2K to allow better sampling for pre-training and 16 or 32 or 48 for fine-tuning
- **How to choose batch size**:
  - Try several values
  - Keep the largest batch that fits without hurting validation performance

Thank you

for your attention